

Lab 5 - Terraform + Ansible + Docker + Kubernetes: kubeadm Cluster Build

Full Lab Guide, Configuration & Source Code

Terraform / Ansible / Docker / Kubernetes Lab Series

Contents

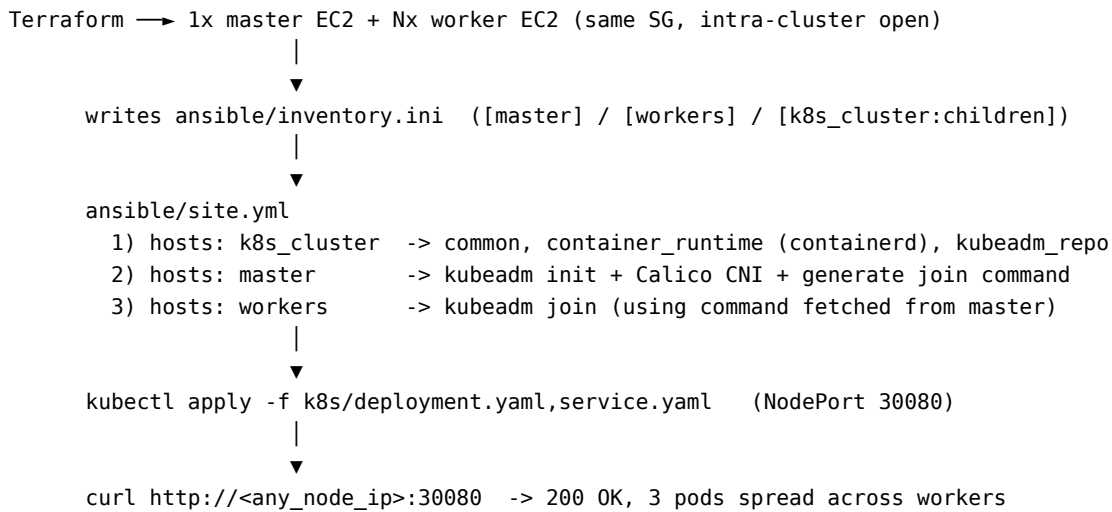
Use Case	1
Architecture	2
Directory Layout	2
Prerequisites	2
Step-by-Step Configuration & Execution	2
Step 1 — Provision master + workers	2
Step 2 — Install required Ansible collection	3
Step 3 — Build the cluster	3
Step 4 — Point local kubectl at the new cluster	3
Step 5 — Deploy the sample application	3
Validation Steps	3
Cleanup	4
Key Concepts Reinforced	4

Appendix: Full Source Code	5
ansible/ansible.cfg	5
ansible/requirements.yml	5
ansible/roles/common/tasks/main.yml	5
ansible/roles/container_runtime/tasks/main.yml	6
ansible/roles/k8s_master/tasks/main.yml	6
ansible/roles/k8s_worker/tasks/main.yml	8
ansible/roles/kubeadm_repo/tasks/main.yml	8
ansible/site.yml	8
k8s/deployment.yaml	9
k8s/service.yaml	10
terraform/main.tf	10
terraform/outputs.tf	12
terraform/variables.tf	12

Use Case

A platform team wants to understand what a managed Kubernetes service (EKS/GKE/AKS) does under the hood by **building a real cluster themselves**: Terraform provisions 1 master + N worker VMs, Ansible turns them into a working kubeadm cluster (containerd runtime, kube packages, control-plane init, Calico CNI, worker join), and finally a sample app is deployed with plain Kubernetes manifests and validated end-to-end.

Architecture



Directory Layout

```

lab5-tf-ansible-docker-k8s-cluster/
├── terraform/
│   ├── main.tf          # master + worker EC2s, SG, inventory.ini generation
│   ├── variables.tf
│   └── outputs.tf
├── ansible/
│   ├── ansible.cfg
│   ├── requirements.yml
│   ├── site.yml         # 3-play orchestration: common -> master -> workers
│   ├── generated/       # join command + kubeconfig fetched here at runtime
│   └── roles/
│       ├── common/      # swap off, kernel modules, sysctl
│       ├── container_runtime/ # containerd + SystemdCgroup fix
│       ├── kubeadm_repo/ # kubelet/kubeadm/kubectl install + hold
│       ├── k8s_master/  # kubeadm init, Calico, join-cmd export
│       └── k8s_worker/  # kubeadm join
└── k8s/
    ├── deployment.yaml # 3-replica nginx demo app
    └── service.yaml    # NodePort 30080
  
```

Prerequisites

- Terraform >= 1.5.0, AWS CLI configured
- Ansible >= 2.14 + ansible-galaxy collection install -r ansible/requirements.yml
- EC2 key pair + local .pem
- kubectl installed locally (for post-deployment validation using the fetched kubeconfig)

Step-by-Step Configuration & Execution

Step 1 — Provision master + workers

```

cd terraform
terraform init
terraform apply \
  
```

```
-var="key_name=my-ec2-keypair" \  
-var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem" \  
-var="worker_count=2"
```

This generates `../ansible/inventory.ini` with `[master]`, `[workers]`, and a `[k8s_cluster:children]` group that ties both together — the private IP of the master (`private_ip`) is passed for `--apiserver-advertise-address`.

Step 2 — Install required Ansible collection

```
cd ../ansible  
ansible-galaxy collection install -r requirements.yml
```

Step 3 — Build the cluster

```
until ansible k8s_cluster -m ping; do sleep 5; done  
ansible-playbook site.yml
```

Runtime is roughly 5–8 minutes. In order: 1. `common + container_runtime + kubeadm_repo` run on **every** node 2. `k8s_master` runs `kubeadm init`, installs Calico, and **fetches** both the join command and admin kubeconfig back to `ansible/generated/` 3. `k8s_worker` copies that join command to each worker and executes it

Step 4 — Point local kubectl at the new cluster

```
export KUBECONFIG=$(pwd)/generated/kubeconfig  
kubectl get nodes -o wide
```

Step 5 — Deploy the sample application

```
kubectl apply -f ../k8s/deployment.yaml  
kubectl apply -f ../k8s/service.yaml
```

Validation Steps

1. All nodes Ready

```
kubectl get nodes  
# Expect: 1 master (control-plane) + N workers, STATUS = Ready
```

2. CNI healthy

```
kubectl get pods -n kube-system | grep calico  
# all calico-node / calico-kube-controllers pods Running
```

3. App pods scheduled and Running

```
kubectl get pods -o wide -l app=lab5-demo-app  
# 3/3 Running, spread across different worker nodes (check NODE column)
```

4. Service reachable via NodePort from ANY node's public IP

```
curl -s -o /dev/null -w "%{http_code}\n" http://<master_or_worker_public_ip>:30080
# Expect: 200 -- NodePort routes to pods regardless of which node you hit
```

5. Self-healing check

```
kubectl delete pod -l app=lab5-demo-app --field-selector=status.phase=Running -l
app=lab5-demo-app --force --grace-period=0 | head -1
kubectl get pods -l app=lab5-demo-app -w
# Deployment controller recreates the deleted pod automatically
```

6. **Idempotent re-run** — re-running `ansible-playbook site.yml` should skip `kubeadm init/kubeadm join` (guarded by the `stat` checks) and report `changed=0` for those tasks.

Cleanup

```
cd terraform
terraform destroy \
  -var="key_name=my-ec2-keypair" \
  -var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem" \
  -var="worker_count=2"
rm -rf ../ansible/generated/*
```

Key Concepts Reinforced

- Full manual `kubeadm` bootstrap: `containerd`, `cgroup` driver alignment, CNI, join tokens
- Using Ansible's `fetch` module to move generated artifacts (`join` command, `kubeconfig`) from a remote master back to the control node for reuse
- `NodePort` services and why they're reachable from every node, not just the one running the pod
- Guarding cluster-mutating commands (`stat` + `when`) so playbooks stay idempotent

Appendix: Full Source Code

Every configuration and code file for this lab is listed below in full, organized by path, so the lab can be reproduced without needing the original repository.

ansible/ansible.cfg

```
[defaults]
inventory = inventory.ini
host_key_checking = False
retry_files_enabled = False
```

ansible/requirements.yml

```
---
collections:
  - name: community.general
    version: ">=8.0.0"
```

ansible/roles/common/tasks/main.yml

```
---
- name: Disable swap (required by kubelet)
  command: swapoff -a
  changed_when: false

- name: Remove swap entry from fstab
  replace:
    path: /etc/fstab
    regexp: '^(#[^#].*\sswap\s.*)$'
    replace: '# \1'

- name: Load required kernel modules
  community.general.modprobe:
    name: "{{ item }}"
    state: present
  loop:
    - overlay
    - br_netfilter

- name: Persist kernel modules across reboot
  copy:
    dest: /etc/modules-load.d/k8s.conf
    content: |
      overlay
      br_netfilter

- name: Set required sysctl params for Kubernetes networking
  sysctl:
    name: "{{ item.name }}"
    value: "{{ item.value }}"
    sysctl_set: true
```

```
state: present
reload: true
loop:
- { name: "net.bridge.bridge-nf-call-iptables", value: "1" }
- { name: "net.bridge.bridge-nf-call-ip6tables", value: "1" }
- { name: "net.ipv4.ip_forward", value: "1" }
```

ansible/roles/container_runtime/tasks/main.yml

```
---
- name: Install containerd prerequisites
  apt:
    name: [ca-certificates, curl, gnupg]
    state: present
    update_cache: true

- name: Install containerd
  apt:
    name: containerd
    state: present

- name: Create containerd config directory
  file:
    path: /etc/containerd
    state: directory

- name: Generate default containerd config
  shell: containerd config default > /etc/containerd/config.toml
  args:
    creates: /etc/containerd/config.toml

- name: Enable SystemdCgroup in containerd config (required for kubelet cgroup driver match)
  replace:
    path: /etc/containerd/config.toml
    regexp: 'SystemdCgroup = false'
    replace: 'SystemdCgroup = true'

- name: Restart and enable containerd
  systemd:
    name: containerd
    state: restarted
    enabled: true
```

ansible/roles/k8s_master/tasks/main.yml

```
---
- name: Check if cluster is already initialized
  stat:
    path: /etc/kubernetes/admin.conf
    register: kubeadm_admin_conf

- name: Initialize the control plane with kubeadm
```

```
command: >
  kubeadm init
  --apiserver-advertise-address={{ private_ip }}
  --pod-network-cidr=192.168.0.0/16
when: not kubeadm_admin_conf.stat.exists
register: kubeadm_init

- name: Create .kube directory for ubuntu user
  file:
    path: /home/ubuntu/.kube
    state: directory
    owner: ubuntu
    group: ubuntu
    mode: "0755"

- name: Copy admin.conf to ubuntu user's kubeconfig
  copy:
    src: /etc/kubernetes/admin.conf
    dest: /home/ubuntu/.kube/config
    remote_src: true
    owner: ubuntu
    group: ubuntu
    mode: "0600"

- name: Install Calico CNI (pod networking)
  become_user: ubuntu
  command: kubectl apply -f
https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/calico.yaml
  environment:
    KUBECONFIG: /home/ubuntu/.kube/config
  when: not kubeadm_admin_conf.stat.exists

- name: Generate the kubeadm join command on the master
  command: kubeadm token create --print-join-command
  register: join_command_raw

- name: Save join command to a local file (fetched by control node)
  copy:
    content: "{{ join_command_raw.stdout }}"
    dest: /tmp/kubeadm_join_command.sh
    mode: "0644"

- name: Fetch join command to Ansible control node
  fetch:
    src: /tmp/kubeadm_join_command.sh
    dest: "{{ playbook_dir }}/generated/kubeadm_join_command.sh"
    flat: true

- name: Fetch admin.conf kubeconfig to Ansible control node (for local kubectl use)
  fetch:
    src: /etc/kubernetes/admin.conf
    dest: "{{ playbook_dir }}/generated/kubeconfig"
    flat: true
```

ansible/roles/k8s_worker/tasks/main.yml

```
---
- name: Check if node has already joined the cluster
  stat:
    path: /etc/kubernetes/kubelet.conf
  register: kubelet_conf

- name: Copy the join command generated by the master
  copy:
    src: "{{ playbook_dir }}/generated/kubeadm_join_command.sh"
    dest: /tmp/kubeadm_join_command.sh
    mode: "0755"
  when: not kubelet_conf.stat.exists

- name: Execute the join command
  command: sh /tmp/kubeadm_join_command.sh
  when: not kubelet_conf.stat.exists
```

ansible/roles/kubeadm_repo/tasks/main.yml

```
---
- name: Add Kubernetes apt GPG key
  get_url:
    url: https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key
    dest: /etc/apt/keyrings/kubernetes-apt-keyring.asc
    mode: "0644"

- name: Add Kubernetes apt repository
  apt_repository:
    repo: "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.asc]
https://pkgs.k8s.io/core:/stable:/v1.30/deb//"
    state: present
    filename: kubernetes

- name: Install kubelet, kubeadm, kubectl
  apt:
    name: [kubelet, kubeadm, kubectl]
    state: present
    update_cache: true

- name: Hold Kubernetes packages at current version
  dpkg_selections:
    name: "{{ item }}"
    selection: hold
  loop: [kubelet, kubeadm, kubectl]
```

ansible/site.yml

```
---
- name: Common OS prep + container runtime on all nodes
  hosts: k8s_cluster
```



```
become: true
roles:
  - common
  - container_runtime
  - kubeadm_repo

- name: Initialize control plane
  hosts: master
  become: true
  roles:
    - k8s_master

- name: Join worker nodes
  hosts: workers
  become: true
  roles:
    - k8s_worker
```

k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lab5-demo-app
  labels:
    app: lab5-demo-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: lab5-demo-app
  template:
    metadata:
      labels:
        app: lab5-demo-app
    spec:
      containers:
        - name: web
          image: nginxdemos/hello:latest
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "50m"
              memory: "64Mi"
            limits:
              cpu: "200m"
              memory: "128Mi"
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 5
```

```
periodSeconds: 10
```

k8s/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: lab5-demo-app-svc
spec:
  type: NodePort
  selector:
    app: lab5-demo-app
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

terraform/main.tf

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
    local = {
      source  = "hashicorp/local"
      version = "~> 2.4"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

data "aws_vpc" "default" { default = true }

data "aws_subnets" "default" {
  filter {
    name     = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
  filter {
    name     = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}
```

```

}

resource "aws_security_group" "k8s" {
  name_prefix = "lab5-k8s-sg-"
  vpc_id      = data.aws_vpc.default.id

  ingress { description = "SSH"          from_port = 22   to_port = 22   protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  ingress { description = "K8s API"      from_port = 6443 to_port = 6443 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  ingress { description = "NodePort range" from_port = 30000 to_port = 32767 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  # Intra-cluster traffic: all protocols open between cluster members (simplify lab networking)
  ingress { description = "Intra-cluster" from_port = 0     to_port = 0     protocol = "-1" self = true }
  egress { from_port = 0 to_port = 0 protocol = "-1" cidr_blocks = ["0.0.0.0/0"] }

  tags = { Name = "lab5-k8s-sg" }
}

resource "aws_instance" "master" {
  ami            = data.aws_ami.ubuntu.id
  instance_type  = var.master_instance_type
  subnet_id      = data.aws_subnets.default.ids[0]
  vpc_security_group_ids = [aws_security_group.k8s.id]
  key_name       = var.key_name
  root_block_device { volume_size = 20 }
  tags = { Name = "lab5-k8s-master" }
}

resource "aws_instance" "worker" {
  count          = var.worker_count
  ami            = data.aws_ami.ubuntu.id
  instance_type  = var.worker_instance_type
  subnet_id      = data.aws_subnets.default.ids[0]
  vpc_security_group_ids = [aws_security_group.k8s.id]
  key_name       = var.key_name
  root_block_device { volume_size = 20 }
  tags = { Name = "lab5-k8s-worker-${count.index}" }
}

resource "local_file" "ansible_inventory" {
  filename = "${path.module}/../ansible/inventory.ini"
  content  = <<-EOT
[master]
  ${aws_instance.master.public_ip} ansible_user=ubuntu ansible_ssh_private_key_file=${var.private_key_path} ansible_
o StrictHostKeyChecking=no' private_ip=${aws_instance.master.private_ip}

[workers]
  %{ for w in aws_instance.worker ~}
  ${w.public_ip} ansible_user=ubuntu ansible_ssh_private_key_file=${var.private_key_path} ansible_ssh_common_args='-
o StrictHostKeyChecking=no' private_ip=${w.private_ip}
  %{ endfor ~}

[k8s_cluster:children]
master
workers
EOT
}

```

terraform/outputs.tf

```
output "master_public_ip" {
  value = aws_instance.master.public_ip
}

output "worker_public_ips" {
  value = aws_instance.worker[*].public_ip
}
```

terraform/variables.tf

```
variable "aws_region"          { default = "us-east-1" }
variable "master_instance_type" { default = "t3.medium" }
variable "worker_instance_type" { default = "t3.medium" }
variable "worker_count"        { default = 2 }
variable "key_name"             { type = string }
variable "private_key_path"     { type = string }
```